

定义

字符串

一个 **字符串** S 是将 n 个字符顺次排列形成的序列, n 称为 S 的长度, 表示为 $|S|$ 。

如果字符串下标从 1 开始计算, S 的第 i 个字符表示为 $S[i]$

如果字符串下标从 0 开始计算, S 的第 i 个字符表示为 $S[i - 1]$ 。

子串

字符串 S 的 **子串** $S[i..j]$, $i \leq j$, 表示 S 串中从 i 到 j 这一段, 也就是顺次排列 $S[i], S[i + 1], \dots, S[j]$ 形成的字符串。

子序列

字符串 S 的 **子序列** 是从 S 中将若干元素提取出来并不改变相对位置形成的序列, 即 $S[p_1], S[p_2], \dots, S[p_k]$, $1 \leq p_1 < p_2 < \dots < p_k \leq |S|$ 。

前缀

前缀 是指从串首开始到某个位置 i 结束的一个特殊子串。字符串 S 的以 i 结尾的前缀表示为 $Prefix(S, i)$, 也就是 $Prefix(S, i) = S[0..i]$ 。

真前缀 指除了 S 本身的 S 的前缀。

后缀

后缀 是指从某个位置 i 开始到整个串末尾结束的一个特殊子串。字符串 S 的从 i 开头的后缀表示为 $Suffix(S, i)$, 也就是 $Suffix(S, i) = S[i..|S| - 1]$ 。

真后缀 指除了 S 本身的 S 的后缀。

字典序

以第 i 个字符作为第 i 关键字进行大小比较, 空字符小于字符集内任何字符 (即: $a < aa$) 。

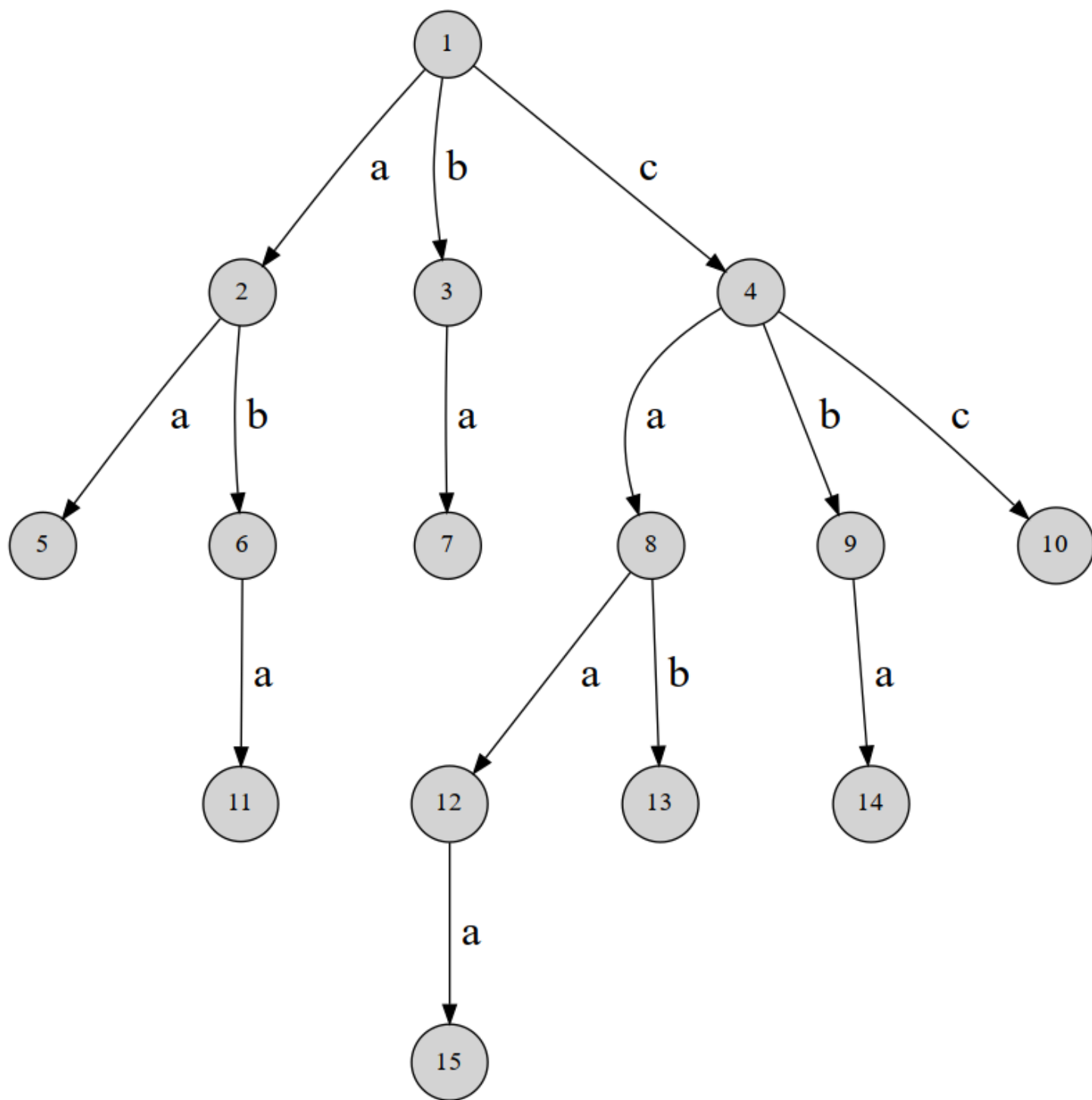
回文串

回文串 是正着写和倒着写相同的字符串，即满足 $\forall 1 \leq i \leq |s|, s[i] = s[|s| + 1 - i]$ 的 s 。

字典树 (Trie)

定义

像字典一样的树（以树的结构来记录字符串）



Trie

实现

```
int cnt=0;
bool vis[1000010];
int tree[1000010][26];
void ins(string s){
    int root=0;
    for(int i=0;i<(int)s.length();i++)
    {
        int id=s[i]-'a';
        if(tree[root][id]==0)
            tree[root][id]=++cnt;
        root=tree[root][id];
    }
    vis[root]=1;
}
int find(string s){
    int root=0;
    for(int i=0;i<(int)s.length();i++)
    {
        int id=s[i]-'a';
        if(tree[root][id]==0)
            return 0;
        root=tree[root][id];
    }
    return 1;
}
```

特点

空间换时间，最坏情况下空间复杂度为**字符串最大长度*字符集大小**。

用处

查找字典中某一字符串

维护字典中的**前缀**

.....

[P2580 于是他错误的点名开始了](#) 板子题

[UVA11732 "strcmp\(\)" Anyone?](#) trie简单应用

[UVA1401 Remember the Word](#) trie+dp

字符串Hash

定义

我们定义一个把字符串映射到整数的函数 f ，这个 f 称为是 Hash 函数。

Hash 的核心思想在于，通过Hash函数字符串映射为一个值域较小，方便比较的范围。

特点

牺牲小小的正确率，实现高效的字符串查询。

1. 在 Hash 函数值不一样的时候，两个字符串一定不一样；
2. 在 Hash 函数值一样的时候，两个字符串不一定一样。

错误率

若进行 n 次比较，每次错误率 $\frac{1}{M}$ ，那么总错误率是 $1 - \left(1 - \frac{1}{M}\right)^n$ 。

实现

$$hash = \sum_{i=1}^n base^{n-i} a_i$$

把字符串看成 $base$ 进制数，并模一个大数解决溢出， $a[i]=s[i]-'a'$ 。

```
//自然溢出
//mod=2^64,好写，容易被卡
#define rep(i,l,r) for(int i=l;i<=r;i++)
typedef unsigned long long ull;
const int N=1e6+5;
const int P=13331;
ull h[N], p[N]; //
void init(string s,int n){
    p[0]=1;
    for(int i=1;i<=n;i++){
        h[i]=h[i-1]*P+s[i];
        p[i]=p[i-1]*P;
    }
}
inline ull get(int l, int r){
    return h[r]-h[l-1]*p[r-l+1];
}
```

```

//单模数
#define rep(i,l,r) for(int i=l;i<=r;i++)
typedef unsigned long long ull;
const int N=1e6+5;
const ull P=13331;
const int mod=1e9+7;
int h[N], p[N];
void init(string s,int n){
    p[0]=1;
    rep(i,1,n){
        h[i]=(h[i-1]*P+(s[i]-'a'+1))%mod;
        p[i]=p[i-1]*P%mod;
    }
}
inline int get(int l,int r){
    return (h[r]-h[l-1]*p[r-(l-1)]%mod+mod)%mod;
}

```

```

//双哈希
#define rep(i,l,r) for(int i=l;i<=r;i++)
typedef pair<int, int> Pair;
const int mod1=3145739;
const int mod2=6291469;
int has1[maxn],has2[maxn];
int bas1[maxn],bas2[maxn];
int p1=2333,p2=17;//如有误差手动修改
void init(string s,int len){
    has1[0]=has2[0]=0;
    bas1[0]=bas2[0]=1;
    for(int i=1;i<=len;++i){
        bas1[i]=(bas1[i-1]*p1)%mod1;
        bas2[i]=(bas2[i-1]*p2)%mod2;
    }
    for(int i=1;i<=len;++i){
        has1[i]=(has1[i-1]*p1+s[i])%mod1;
        has2[i]=(has2[i-1]*p2+s[i])%mod2;
    }
}
int get1(int l,int r){//返回以r结尾长度len的子串has1值
    return ((has1[r]-has1[l-1]*bas1[r-(l-1)])%mod1+mod1)%mod1;
}
int get2(int l,int r){//返回以r结尾长度len的子串has2值
    return ((has2[r]-has2[l-1]*bas2[r-(l-1)])%mod2+mod2)%mod2;
}
Pair get(int l,int r){
    return {get1(l,r),get2(l,r)};
}

```

应用

字符串匹配

求出模式串的哈希值后，求出文本串每个长度为模式串长度的子串的哈希值，分别与模式串的哈希值比较即可。

判断回文串

对原串做一次反向哈希，判断正向哈希值与反向哈希值是否相同

求最长回文子串

二分答案，判断是否可行时枚举回文中心（对称轴），哈希判断两侧是否相等。需要分别预处理正着和倒着的哈希值。时间复杂度 $O(n \log n)$ 。

Tip：此问题可以使用Manacher算法在 $O(n)$ 的时间内解决。

[P3370 【模板】字符串哈希](#) 板子题

[P3805 【模板】manacher 算法](#) 哈希+二分解决

[P3538 \[POI2012\]OKR-A Horrible Poem](#) 哈希处理字符串循环节+分解质因数

[P4503 \[CTSC2014\] 企鹅 QQ](#) 删去一位的哈希比较

KMP

一个暴力的字符串匹配

从主串 S 的第一个字符开始和模式串 T 的第一个字符进行比较，若相等，则继续比较二者的后续字符；否则，模式串 T 回退到第一个字符，重新和主串 S 的第二个字符进行比较。如此往复，直到 S 或 T 中所有字符比较完毕。

```

// 暴力匹配
int i = 0, j = 0;
while (i < s.length())
{
    if (s[i] == p[j])
        ++i, ++j;
    else
        i = i - j + 1, j = 0;
    if (j == p.length()) // 匹配成功
    {
        // 对s[i - j .. i - 1]进行一些操作
        cout << i - j << endl;
        i = i - j + 1;
        j = 0;
    }
}

```

↓

abababcbabaa

ababcbabaa

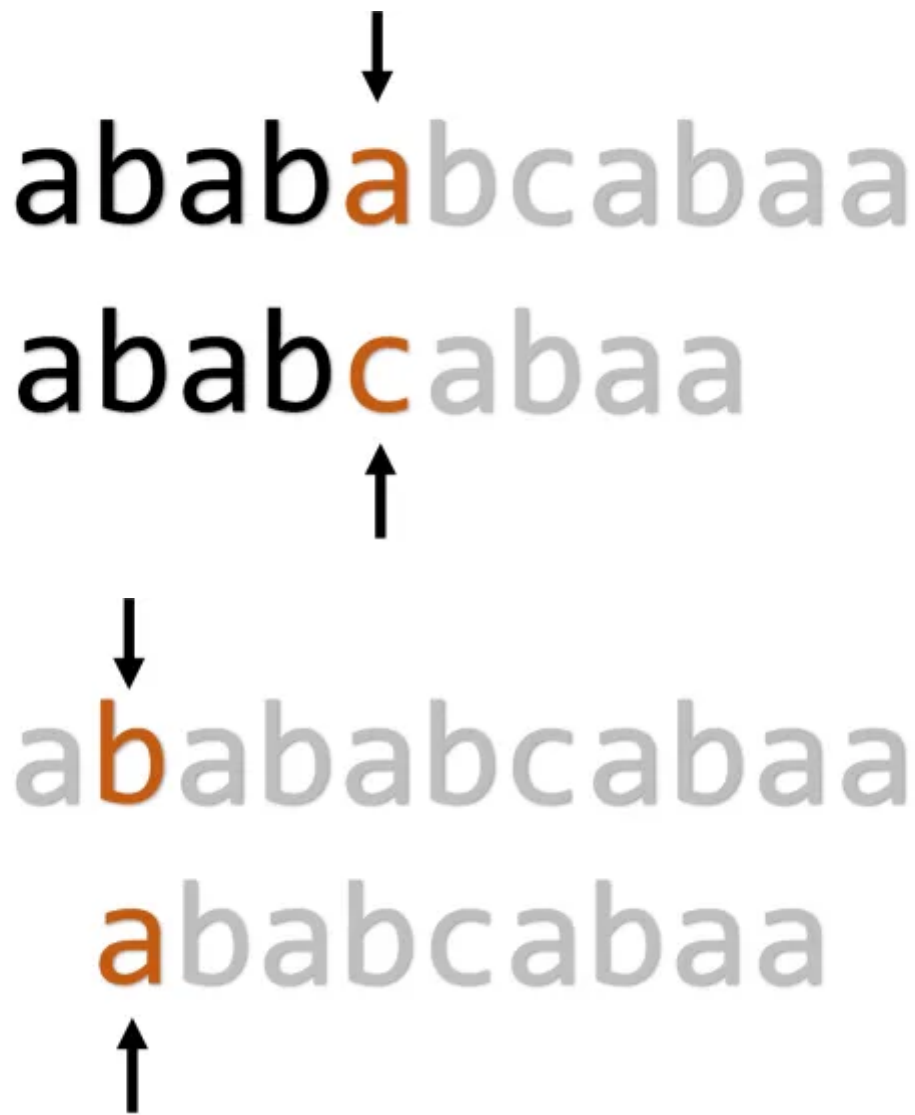
↑

↓

abababcbabaa

ababcbabaa

↑



设两个字符串的长度分别为 n 和 m ，则暴力匹配的最坏时间复杂度是 $O(nm)$ 。

缺点

在匹配不成功时回溯后从头来过重新匹配，浪费太多时间。

改进

border

对字符串 s 和 $0 \leq r < |s|$ ，若 s 长度为 r 的前缀和长度为 r 的后缀相等，即**真前缀和真后缀公共部分的最大长度**，就称 s 长度为 r 的前缀是 s 的 border。

next数组

$next[i]$ 表示字符串前缀 $S[0..i]$ 的border。

以下为字符串“ababcabaa”对应的next数组

0	1	2	3	4	5	6	7	8
a	b	a	b	c	a	b	a	a
0	0	1	2	0	1	2	3	1

有什么用？

回到上文中暴力匹配第一次失配的地方

Diagram illustrating a string matching failure. The top string is "abababcabaa" and the bottom string is "ababcabaa". The first four characters "abab" match. At index 4, the top string has 'a' and the bottom string has 'c', which do not match. Arrows point to these mismatching characters.

这时 S 中的 'a' 与 p 中的 'c' 没有配上，我们计划保持 i 指针（上面的指针）不变，而把 j 指针左移。注意到，"abab" 已经匹配成功了，“abab”的border为2，即 $next[3]=2$ 。所以我们可以把这个"ab"利用起来，变成下面这样

Diagram illustrating the string matching process after a jump. The top string is "abababcabaa" and the bottom string is "abababcabaa". The first two characters "ab" match. At index 2, the top string has 'a' and the bottom string has 'a', which match. Arrows point to these matching characters.

即 $j = next[j] - 1$, 相比暴力匹配下 j 的回溯省去了很多无用的失败匹配。

再来看个例子：

↓
ABACABA**B**ACABAD

ABACABAD**D**

↑

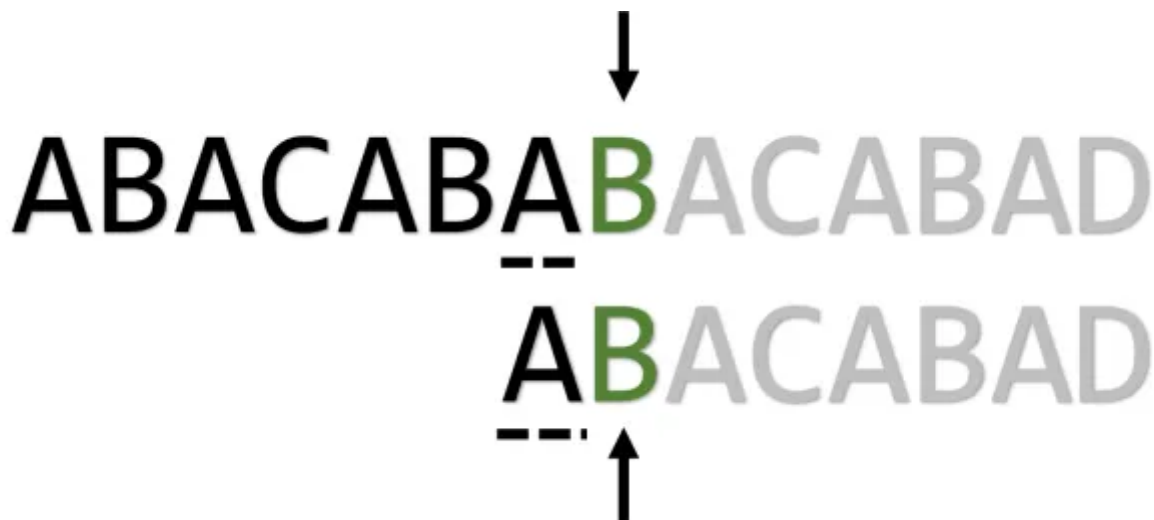
↓
ABACABA**B**ACABAD

ABAC**C**ABAD

↑

↓
ABACABA**B**ACABAD
--

ABAC**C**ABAD
--
↑



以上过程用代码表达：

```
for (int i = 0, j = 0; i < s.length(); ++i)
{
    while (j && s[i] != p[j]) j = nxt[j - 1]; // 不断前移j指针，直到成功匹配或移到头为止
    if (s[i] == p[j]) j++; // 当前位匹配成功，j指针右移
    if (j == p.length())
    {
        // 对s[i - j + 1 .. i]进行一些操作
        j = nxt[j - 1];
    }
}
```

next数组怎么求？

暴力: $O(m^2)$

一种精妙的做法：在**错开**一位后，让p自己匹配自己（这相当于是用前缀去匹配后缀）。我们知道 $\text{next}[0]=0$ ，而之后的每一位则可以通过在匹配过程中记录j值得到。

还是以刚刚的模式串为例：

↓
ababcaaa
↑
ababcaaa

0

↓
ababcaaa
- - -
↑
ababcaaa

0 0

ababcaabaa
ababcaabaa

0 0 1

ababcaabaa
ababcaabaa

0 0 1 2

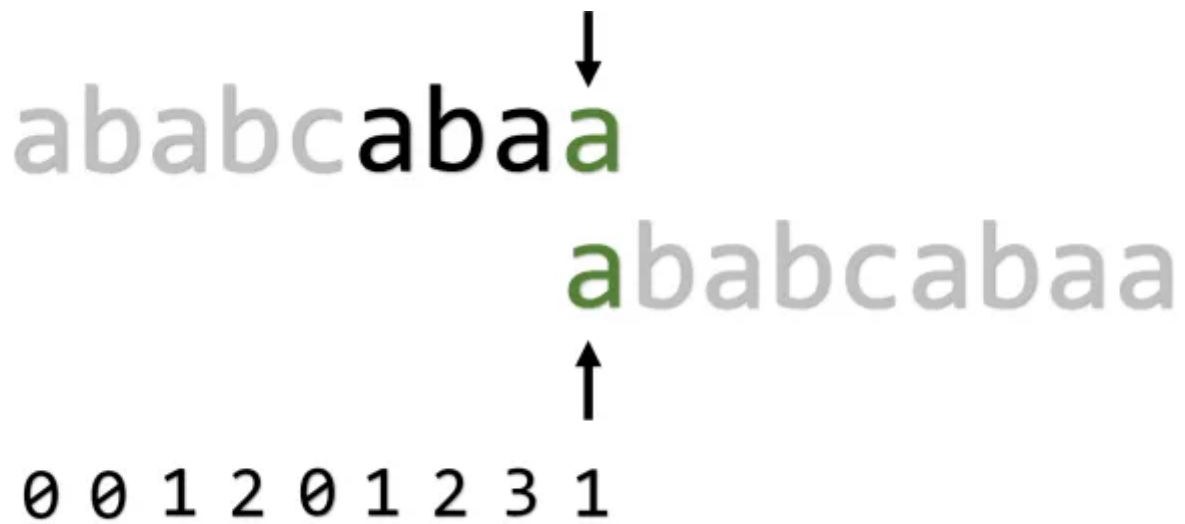
abab**c**abaa
a**b**abababaa

0 0 1 2

如此继续...

ababcbaba**a**
abab**b**ababaa

0 0 1 2 0 1 2 3



代码表示：

```
// nxt[0] = 0;
for (int i = 1, j = 0; i < len; ++i)
{
    while (j && p[i] != p[j]) j = nxt[j - 1];
    if (p[i] == p[j]) j++;
    nxt[i] = j;
}
```

至此，我们完成了`next`数组的线性求解，以及通过`next`数组实现线性时间内的字符串匹配，以上就是所谓的`KMP`算法。

板子题：[P3375](#) 【模板】KMP字符串匹配

完整代码：


```

#include <bits/stdc++.h>
using namespace std;
const int MAXN = 1e6 + 5;
int nxt[MAXN];
void get_nxt(const string& s) {
    for (int i = 1, j = 0; i < s.length(); ++i) {
        while (j && s[i] != s[j]) j = nxt[j - 1];
        if (s[i] == s[j]) j++;
        nxt[i] = j;
    }
}
void kmp(const string& s, const string& p) {
    for (int i = 0, j = 0; i < s.length(); ++i) {
        while (j && s[i] != p[j]) j = nxt[j - 1];
        if (s[i] == p[j]) j++;
        if (j == p.length()) {
            cout << i - j + 2 << '\n'; // 因为要1-index, 所以是+2
            j = nxt[j - 1];
        }
    }
}
int main() {
    ios::sync_with_stdio(false);
    string s, p;
    cin >> s >> p;
    get_nxt(p);
    kmp(s, p);
    for (int i = 0; i < p.length(); ++i)
        cout << nxt[i] << ' ';
    return 0;
}

```

字符串的周期

对字符串 s 和 $0 < p \leq |s|$, 若 $s[i] = s[i + p]$ 对所有 $i \in [0, |s| - p - 1]$ 成立, 则称 p 是 s 的周期。

对字符串 s 和 $0 \leq r < |s|$, 若 s 长度为 r 的前缀和长度为 r 的后缀相等, 就称 s 长度为 r 的前缀是 s 的 border。

由 s 有长度为 r 的 border 可以推导出 $|s| - r$ 是 s 的周期。

所以根据前缀函数可以在 $O(n)$ 的时间内计算出 s 所有的周期。其中, 由于 $\pi[n - 1]$ 是 s 最长 border 的长度, 所以 $n - \pi[n - 1]$ 是 s 的最小周期。

[CF126B Password](#) kmp的简单运用

[P4391 \[BOI2009\]Radio Transmission](#) 无线传输 最简单的周期

[UVA10298 Power Strings](#) 难一点的周期

[P3435 \[POI2006\] OKR-Periods of Words](#) 再难一点的周期

[P2375 \[NOI2014\] 动物园](#) kmp的变化

[P5829 【模板】失配树](#) kmp+lca

[P4696 \[CEOI2011\] Matching](#) kmp特殊匹配的应用

Manacher(马拉车)

问题

给定一个长度为 n 的字符串 s ，请找到所有对 (i, j) 使得子串 $s[i \dots j]$ 为一个回文串。

暴力: $O(n^2)$

哈希+二分: $O(n \log n)$

Manacher: $O(n)$

介绍

Manacher算法，俗称马拉车，是一种与回文子串有关的算法。

回文串根据其长度地奇偶性可以分为**奇回文串**和**偶回文串**两种，其中奇回文串通常比偶回文串好处理，因为它有一个唯一的中心，而且可以从中心向两边一步一步拓展得到。

对于一个字符串的每个位置 $i = 0 \dots n - 1$ ，我们找出值 $d_1[i]$ 和 $d_2[i]$ 。二者分别表示以位置 i 为中心的、长度为奇数和长度为偶数的回文串个数。换个角度，二者也表示了以位置 i 为中心的最长回文串的半径长度（半径长度 $d_1[i]$ ， $d_2[i]$ 均为从位置 i 到回文串最右端位置包含的字符个数）。

不过，我们对字符串进行一些预处理后，就可以用处理奇回文串的方法处理偶回文串了。方法是：给定一个长度为 n 的字符串 s ，我们在其 $n + 1$ 个空中插入分隔符 $\#$ ，从而构造一个长度为 $2n + 1$ 的字符串 s' 。举例来说，对于字符串 $s = \text{abbab}$ ，其对应的 $s' = \#a\#b\#b\#a\#b\#$ 。

对于字母间的 $\#$ ，其实际意义为 s 中对应的「空」。而两端的 $\#$ 则是为了实现的方便。可以发现，原来的奇回文子串会变成以普通字符为中心的奇回文子串；原来的偶回文子串会变成以 $\#$ 为中心的奇回文子串，此时的 d_1 数组和 d_2 数组可以合并为 1 个数组 d 。

Manacher算法的作用，就是 $O(n)$ 地求出这个 d 数组。

过程

我们维护 mx 和 id ， id 表示回文串的中心， mx 表示回文串右边最大半径下标，有 $mx = id + d[id]$ 。

如果 $mx > i$, 则 $d[i] = \min(d[2 * id - i], mx - i)$; 否则, $d[i] = 1$ 。 ($2 * id - i$ 表示 i 关于 id 对称的坐标点 j 。因为 j 到 id 之间到距离等于 id 到 i 之间到距离 $id - j = i - id$, 所以 $j = 2 * id - i$ 。)

- 在 $mx > i$ 下, 若 $d[j] < mx - i$, 表示以 $S[j]$ 为中心的回文子串包含在以 $S[id]$ 为中心的回文子串中, 由于 i 和 j 对称, 以 $S[i]$ 为中心的回文子串必然包含在以 $S[id]$ 为中心的回文子串中, 所以必有 $d[i] = d[j]$ 。
- 在 $mx > i$ 下, 若 $d[j] \geq mx - i$ 表示以 $S[j]$ 为中心的回文子串不一定完全包含于以 $S[id]$ 为中心的回文子串中, 也就是说 $d[j]$ 表示的回文串半径超过 mx 对称点的坐标, 此时不能利用对称性, 但一定可以扩展到 mx 。
- 在 $mx \leq i$ 的前提下, $d[i] = 1$, 此时暴力扩展。

```
//字符串下标默认为1 即s=' '+s
int d[2*maxn];
int d1[maxn];
int d2[maxn];
#define rep(i,l,r) for(int i=l;i<=r;i++)
void mlc (string s) {
    string st="##";
    rep(i,1,(int)s.length()-1)st+=s[i],st+=" ";
    int mx=0,id=0;
    rep(i,1,(int)st.size()-1){
        d[i]=mx>i?min (d[2*id-i],mx-i):1;
        while (st[i+d[i]]==st[i-d[i]]) d[i]++;
        if (mx<i+d[i]) mx=i+d[i],id=i;
    }
    rep(i,1,(int)st.length()-1){
        if(i&1) d2[i/2]=d[i]/2;
        else d1[i/2]=d[i]/2;
    }
    //s=' '+s
    //i为偶数s[i/2]为中心,对应d1[i];
    //i为奇数(/2,i/2+1)为中心,对应d2[i];
}
```

P3805 【模板】manacher 算法 板子题

P1659 [国家集训队]拉拉队排练 简单运用

P3501 [POI2010]ANT-Antisymmetry01异或意义下的马拉车

P5446 [THUPC2018]绿绿和串串 思维